

## The data use rules

- Participant-level data (one row per person) stays in the Workbench: no copying out, downloading, or screenshots.
- No count of 1–20 participants may be shared, directly or by calculation (differences, percentages). 0 is allowed. Allele counts under 40 are shown as <40.
- Downloads are monitored; participant-level downloads trigger an egress alert.
- Nothing in a GitHub repository (code, comments, outputs) may contain participant data or counts under 20.
- Notify the program 2 weeks before publishing or presenting results.

Articles at [support.researchallofus.org/hc/en-us/articles/number](https://support.researchallofus.org/hc/en-us/articles/number): Data User Code of Conduct 22346176432532; Data and Statistics Dissemination Policy 22346276580372; Egress Alert Policy 4407354684052; Large Download Requests 48941559021716; Publication and Presentation Policy 22346271543828; Researcher FAQ 52861503885460. A workable habit: keep person-level files in one folder that never leaves the workspace, and aggregate outputs in another that is checked before any download.

## Moving around the terminal

|                                 |                                         |
|---------------------------------|-----------------------------------------|
| <code>pwd</code>                | print the folder you are in             |
| <code>ls -lh</code>             | list files with sizes (h = human units) |
| <code>ls -lt   head</code>      | newest files first                      |
| <code>cd my_project</code>      | move into a folder                      |
| <code>cd .. / cd ~</code>       | up one level / home folder              |
| <code>mkdir -p results</code>   | make a folder (no error if it exists)   |
| <code>cp a b / mv a b</code>    | copy / move or rename                   |
| <code>rm file</code>            | delete a file (no trash can – careful)  |
| <code>du -sh data</code>        | total size of a folder                  |
| <code>df -h .</code>            | free disk space on this VM              |
| <code>history   tail -20</code> | your last 20 commands                   |

Tab completes file names. Up-arrow recalls earlier commands. Ctrl-C stops a running command.

## Looking inside a text file

|                                                                                                                                                                                |                                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| <code>head -n 3 file.tsv</code>                                                                                                                                                | first 3 lines (always look first)      |
| <code>tail -n 3 file.tsv</code>                                                                                                                                                | last 3 lines                           |
| <code>less -S file.tsv</code>                                                                                                                                                  | scroll through; q quits; -S no wrap    |
| <code>wc -l file.tsv</code>                                                                                                                                                    | count lines (rows + 1 header)          |
| <code>column -t file.tsv   head</code>                                                                                                                                         | line up columns for reading            |
| <code>head -1 file.tsv   tr '\t' '\n'   cat -n</code>                                                                                                                          | number the columns of the header       |
| <code>cut -f 2,5 file.tsv   head</code>                                                                                                                                        | keep columns 2 and 5 (tab-separated)   |
| <code>cut -d, -f 2 file.csv   head</code>                                                                                                                                      | same, comma-separated                  |
| <code>grep -c 'Female' file.tsv</code>                                                                                                                                         | count lines containing a word          |
| <code>grep -n 'NA' file.tsv   head</code>                                                                                                                                      | show matching lines, with line numbers |
| <code>sort   uniq -c</code>                                                                                                                                                    | count each distinct value (after cut)  |
| <code>awk -F'\t' 'NR&gt;1{s+=\$2} END{print s/(NR-1)}' f.tsv</code>                                                                                                            | mean of column 2, skipping the header  |
| <code>zcat file.gz   head</code>                                                                                                                                               | look inside a gzipped file             |
| Example, a frequency table of column 5 of a phenotype file:                                                                                                                    |                                        |
| <code>cut -f 5 pheno.tsv   sort   uniq -c</code> . The sort is required: <code>uniq</code> only merges adjacent lines. Output stays on your screen; it is not a file to share. |                                        |

## Settings and environment variables

|                                                       |                                                     |
|-------------------------------------------------------|-----------------------------------------------------|
| <code>source config.sh</code>                         | load a project's settings file into this terminal   |
| <code>echo \$CDR_DATASET</code>                       | v9 CDR:<br><b>wb-silky-artichoke-2408.C2025Q4R6</b> |
| <code>echo \$WORKSHOP_BUCKET \$BILLING_PROJECT</code> | bucket and billing project                          |
| <code>env   grep -i -E 'workspace cdr google'</code>  | Workbench variables, if set                         |

The Workbench can define `$CDR_DATASET`, `$WORKSPACE_BUCKET`, `$GOOGLE_PROJECT`, and `$CDR_STORAGE_PATH`, but they are not always set in a new terminal (the “Getting Started” featured-workspace notebooks set them up). A project's own settings file (`export NAME=value`, one per line, sourced by every script) does not depend on them. Set one value for one run with `NAME=value bash script.sh`.

## The CDR: BigQuery from the terminal (bq)

|                                                                                            |                                                              |
|--------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| <code>bq ls \$CDR_DATASET   head -40</code>                                                | list the CDR tables                                          |
| <code>bq show -schema -format=prettyjson \$CDR_DATASET.person</code>                       | column names and types of one table                          |
| <code>bq query -use_legacy_sql=false -dry_run 'SELECT ...'</code>                          | report bytes a query would scan; runs nothing, costs nothing |
| <code>bq query -use_legacy_sql=false -maximum_bytes_billed=25000000000 'SELECT ...'</code> | run with a hard cost cap                                     |
| <code>bq query ... -format=csv -max_rows=10000000 'SELECT ...' &gt; out.csv</code>         | save results as CSV (person-level: keep it in the workspace) |

Table names go in backticks inside SQL: FROM ``$CDR_DATASET.measurement``. Look up concept IDs in the public [Data Browser](#).

### Handy queries (aggregate only)

```
SELECT COUNT(DISTINCT person_id) FROM ` $CDR_DATASET.measurement `
WHERE measurement_source_concept_id = 903133
SELECT concept_id, concept_name, vocabulary_id
FROM ` $CDR_DATASET.concept `
WHERE LOWER(concept_name) LIKE '%height%' LIMIT 20
```

## Cloud storage (gcloud storage; gsutil is the older tool)

|                                                                       |                                                                                                           |
|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>gcloud storage ls \$WORKSHOP_BUCKET</code>                      | list a bucket or folder                                                                                   |
| <code>gcloud storage ls -l gs://.../file</code>                       | list with size                                                                                            |
| <code>gcloud storage du -s -r gs://.../folder</code>                  | total size of a cloud folder (-r readable units)                                                          |
| <code>gcloud storage cat gs://.../file.tsv   head -3</code>           | peek at a cloud text file without copying it                                                              |
| <code>gcloud storage cp gs://.../file lab_data/</code>                | copy cloud → VM disk                                                                                      |
| <code>gcloud storage cp results/x.png \$WORKSHOP_BUCKET/lab/</code>   | copy VM disk → a workspace bucket                                                                         |
| <code>gcloud storage cp -r folder \$WORKSHOP_BUCKET/lab/</code>       | copy a whole folder                                                                                       |
| <code>gcloud -billing-project=\$BILLING_PROJECT storage cp ...</code> | <b>requester-pays</b> buckets (e.g., the controlled dataset bucket): name who pays, before <b>storage</b> |

Older equivalents: `gsutil ls`, `gsutil cp`, `gsutil -u $BILLING_PROJECT cp ...` (requester pays). An error mentioning “requester pays” or “billing” means the billing flag is missing.

## Controlled Tier genomic data (v9)

Root: `gs://vwb-aou-datasets-controlled/v9/`. Below, `wgs/` means `wgs/short_read/snpindel/`. From the v9 Controlled Tier data dictionary; list a folder with `gcloud storage ls` for exact file names. All copies need the billing flag.

|                                                                            |                                                                                                                             |
|----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <code>wgs/acaf_threshold/plink_bed/</code>                                 | whole-genome variants with allele frequency >1% or allele count >100; PLINK 1, one set per chromosome (chr22 .bed ≈ 250 GB) |
| <code>wgs/acaf_threshold/pgen/</code> <code>bgen/</code> <code>vcf/</code> | same variants, other formats                                                                                                |
| <code>wgs/exome/</code> <code>wgs/clinvar/</code>                          | smaller callsets, same layout                                                                                               |
| <code>wgs/aux/ancestry/ancestry_preds.tsv</code>                           | predicted ancestry, 16 PCs per person ( <b>research_id</b> = <b>person_id</b> )                                             |
| <code>wgs/aux/relatedness/</code>                                          | related pairs (kinship > 0.1); samples to drop for an unrelated set                                                         |
| <code>microarray/plink/</code>                                             | genotyping-array data, PLINK 1, all chromosomes in one set                                                                  |

Variant IDs in these files are chr:pos:ref:alt (GRCh38), not rsIDs, and FID is 0. All of Us publishes no HapMap3 subset: make one with `-extract` on a list of IDs matched by position and alleles, then `-update-name` to rsIDs (the .pvar may be .zst-compressed: `plink2 -zst-decompress`).

## PLINK 2: looking at genotype files

|                                                                    |                                                                      |
|--------------------------------------------------------------------|----------------------------------------------------------------------|
| <code>head -n 2 f.fam</code>                                       | people: FID IID father mother sex pheno                              |
| <code>head -n 2 f.bim</code>                                       | variants: chr ID cM position allele1 allele2                         |
| <code>wc -l &lt; f.fam; wc -l &lt; f.bim</code>                    | number of people; number of variants                                 |
| <code>cut -f 1 f.bim   uniq -c</code>                              | variants per chromosome                                              |
| <code>plink2 -bfile f -freq -out out/f</code>                      | allele frequencies → <b>.afreq</b>                                   |
| <code>plink2 -bfile f -missing -out out/f</code>                   | missing-call rates per person / variant                              |
| <code>plink2 -bfile f -extract ids.txt -make-bed -out out/g</code> | keep only listed variant IDs                                         |
| <code>plink2 -bfile f -keep ppl.txt -make-bed -out out/g</code>    | keep only listed people (IID, or FID IID)                            |
| <code>plink2 -pgen f.pgen -pvar f.pvar -psam f.psam ...</code>     | name the three input files separately (e.g. a mounted genotype file) |
| <code>plink2 -bfile f -score w.txt 1 2 3 header -out out/s</code>  | polygenic score from a weight file                                   |
| <code>grep -E 'processed skipped' out/s.log</code>                 | how many weights matched                                             |

.bed is binary: never open it with head or less. PLINK writes a .log next to every output; read it.

## Git: getting code onto the VM and keeping it current

|                                                         |                                                |
|---------------------------------------------------------|------------------------------------------------|
| <code>git clone https://github.com/USER/REPO.git</code> | copy a repository onto the VM                  |
| <code>git pull</code>                                   | fetch and apply the latest changes             |
| <code>git status</code>                                 | what changed; what is untracked or ignored     |
| <code>git diff</code>                                   | line-by-line changes not yet committed         |
| <code>git log -oneline   head</code>                    | recent commits                                 |
| <code>git rev-parse -short HEAD</code>                  | the commit you are on (record it with results) |
| <code>git add scripts/x.py</code>                       | stage a file you intend to commit              |
| <code>git commit -m "Describe the change"</code>        | record staged changes                          |
| <code>git push</code>                                   | send commits to GitHub (needs a token)         |
| <code>git check-ignore -v config.sh</code>              | confirm a file is ignored, and by which rule   |
| <code>git ls-files</code>                               | list every file git tracks                     |
| <code>git restore file</code>                           | discard uncommitted edits to a file            |

## A .gitignore for Workbench projects

```

config.sh  # filled settings
data/      # copied-in data
work/      # person-level files
results/   # outputs (review before sharing)
.ipynb_checkpoints/

```

Commit code, templates, and documentation only. Before committing a notebook, clear its outputs (JupyterLab: Edit → Clear All Outputs): a saved .ipynb stores its outputs. Check with `git diff -cached` before every `git push`.

## GitHub CLI (gh) – on your own computer

|                                                  |                            |
|--------------------------------------------------|----------------------------|
| <code>gh auth login</code>                       | sign in once               |
| <code>gh repo create NAME -private -clone</code> | new repository             |
| <code>gh repo clone USER/REPO</code>             | clone by short name        |
| <code>gh repo view -web</code>                   | open the repo in a browser |

## Getting aggregate results to your computer

1. Run your disclosure check on the output folder (no count of 1–20, directly or by subtraction; no person-level file). It must pass.
2. Open each file in **results/** and review it yourself.
3. In the JupyterLab file browser, right-click the file → **Download**.

Never download person-level files, copied-in data, or an archive that contains them.